COMPUTER SYSTEMS AND PLATFORM TECHNOLOGIES

IT0047

FEU ALABANG FEU DILIMAN FEU TECH

Technology Driven by Innovation

MODULE 5

8086/8088 Instruction Sets

MODULE 5.2 Part 1

Logical, Shift and Rotate Instructions

At the end of the lesson, the student should be able to :

- **Identify the different Logical, Shift and Rotate Instruction sets**
- **Identify the value of the destination operand and conditional flags for each logical, shirt or rotate instruction sets**

Logic Instructions

- Logic instructions include the logic operations AND, OR, XOR, and NOT. Also included in this group is the TEST instruction.
- The TEST instruction is somewhat similar to the CMP instruction where the flags are modified but the operands are not altered.

AND Instruction

- The AND (***Logical AND***) instruction logically ANDs the source and the destination operands and stores the result in the destination operand.

Format: AND D, S

Action: D [D] . [S]

Valid Format

Destination	Source	Example
register	register	AND BX, CX
register	MM	AND DX, [BP + SI]
MM	register	AND BETA, CX
register	immediate	AND BX, 0015H
MM	immediate	AND byte ptr BETA, 12H

Pointers:

1. The AND instruction does not allow both source and destination operands to be memory locations (no memory to memory AND). Therefore, the instruction **AND SET, FILE** is invalid.

2. Both source and destination operands should be of the same data size. The instruction **AND AH, BX** is therefore invalid.

3. The destination operand cannot be immediate data. Therefore, the instruction **AND 1800H, CX** is invalid.

4. CF and OF are always 0 after execution while AF is undefined.

5. As with the MOV instruction, if the destination operand is a memory location and the source operand is immediate data, the prefix ***byte ptr*** or ***word ptr*** should appear after the AND instruction to denote the data size of the immediate operand.

Continuation

6. The AND instruction can be used to clear out or zero out certain bits in a byte or word. In order to do this, simply logically AND a ***bit mask pattern*** and the byte or word concerned.

A bit mask pattern contains a pattern of 1's and 0's. A 0 in a bit position will cause that bit position to be zeroed out, while a 1 in a bit position will not change the value.

Example:

MOV AL, 05H

MOV BL, FEH

AND AL, BL

AL = 0 0 0 0 0 1 0 1

(mask) BL = 1 1 1 1 1 1 1 0

AL = 0 0 0 0 0 1 0 0

bit 0 is zeroed out, the rest of the bits remain the same.

Flags:

CF, OF, PF, ZF, SF, are all equal to 0. AF is undefined.

OR Instruction

- The OR (***Logical OR***) instruction logically ORs the source and the destination operands and stores the result in the destination operand. Format: OR D, S

Action: D ← [D] + [S]

Destination	Source	Example
register	register	OR BX, CX
register	MM	OR DX, [BP + SI]
MM	register	OR BETA, CX
register	immediate	OR BX, 0015H
MM	immediate	OR byte ptr BETA, 12H

Pointers:

1. The OR instruction does not allow both source and destination operands to be memory locations (no memory to memory OR). **OR SET, FILE** is therefore invalid.

2. Both source and destination operands should be of the same data size. Therefore, the instruction **OR AH, BX** is invalid.

3. The destination operand cannot be immediate data. Thus, the instruction **OR 1800H, CX** is invalid.

4. CF and OF are always 0 after execution while AF is undefined.

continuation

5. As with the MOV instruction, if the destination operand is a memory location and the source operand is immediate data, the prefix ***byte ptr*** or ***word ptr*** should appear after the OR instruction to denote the data size of the immediate operand.

6. The OR instruction can be used to set a certain bit to 1 in a byte or word. In order to do this, simply logically OR a ***bit mask pattern*** and the byte or word concerned. A bit mask pattern contains a pattern of 1's and 0's. A 1 in a bit position will cause that bit position to be set to 1, while a 0 in a bit position will not change the value.

Example:

MOV CL, 05H

MOV DL, 80H

OR CL, DL CL

= 0 0 0 0 0 1 0 1

(mask) DL = 1 0 0 0 0 0 0 0

CL = 1 0 0 0 0 1 0 1

bit 7 is set to 1, the rest of the bits remain the same. Flags: CF, OF , PF, and ZF are all equal to 0. SF = 1 while AF is undefined.

XOR Instruction

- The XOR (***Logical XOR***) instruction logically XORs the source and the destination operands and stores the result in the destination operand.

Format: XOR D, S

Action: D ← [D] + [S]

Destination	Source	Example
register	register	XOR BX, CX
register	MM	XOR DX, [BP + SI]
MM	register	XOR BETA, CX
register	immediate	XOR BX, 0015H
MM	immediate	XOR byte ptr BETA, 12H

Pointers:

1. The XOR instruction does not allow both source and destination operands to be memory locations (no memory to memory XOR). **XOR SET, FILE** is therefore invalid.

2. Both source and destination operands should be of the same data size. The instruction **XOR AH, BX** is invalid.

3. The destination operand cannot be immediate data. Thus, the instruction **XOR 1800H, CX** is invalid.

continuation

4. CF and OF are always 0 after execution while AF is undefined.

5. As with the MOV instruction, if the destination operand is a memory location and the source operand is immediate data, the prefix ***byte ptr*** or ***word ptr*** should appear after the XOR instruction to denote the data size of the immediate operand.

6. It is faster for the 8086/8088 to initialize a register to zero using the XOR instruction rather than the MOV instruction. Since any value XORed to itself to zero, **XOR AL, AL** is faster to execute than **MOV AL, 00H**.

NOT Instruction

The NOT (***Logical NOT***) instruction performs a 1's complement on the operand.

Format: NOT D

Action: D ← [D]

Destination	Example
register	NOT AX
MM	NOT byte ptr BETA

Pointers:

1. The destination operand cannot be immediate data. Thus, the instruction **NOT 1800H** is invalid.

2. Flags are not affected.

3. If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the NOT instruction to denote the data size of the destination operand.

TEST Instruction

- The TEST instruction logically ANDs the source and the destination operands. The result is discarded but the values of the status flags are updated.

Format: TEST D, S Action: [D] . [S]

Pointers:

1. The TEST instruction does not allow both source and destination operands to be memory locations (no memory to memory AND). Therefore, the instruction **TEST SET, FILE** is invalid.

2. Both source and destination operands should be of the same data size. The instruction **TEST AH, BX** is therefore invalid.

3. CF and OF are always 0 after execution while AF is undefined.

4. As with the MOV instruction, if the destination operand is a memory location and the source operand is immediate data, the prefix ***byte ptr*** or ***word ptr*** should appear after the TEST instruction to denote the data size of the immediate operand.

MODULE 5.2 Part 2

Basic 8086/8088 Instruction Sets: Shift Instructions

At the end of the lesson, the student should be able to :

- **Identify the different Logical Instruction sets**
- **Identify the value of conditional flags for each logical instruction sets**

SHIFT INSTRUCTIONS

- Shift instructions are instructions that also perform bit manipulation.
- Shift Instructions can be classified as either ***logical shift*** or ***arithmetic shift***.
- Shift instructions can also be classified by the direction of the shift (i.e., ***shift to the left*** or ***shift to the right***).

Four types of shift instructions:

- ***shift logical left*** (SHL),
- ***shift logical right*** (SHR),
- ***shift arithmetic left*** (SAL),
- ***shift arithmetic right*** (SAR).

Note:

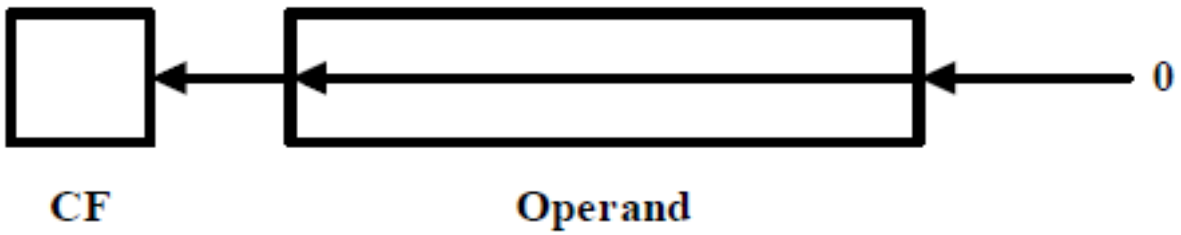
Logical shifts are used mostly to isolate bits in bytes or words, while arithmetic shifts are used mostly to multiply and divide binary numbers by powers of two.

The SHL/SAL Instructions

- SHL (***Shift Logical Left***) and SAL (***Shift Arithmetic Left***) are two different mnemonics but perform the same operation. Both instructions shift to the left the destination operand by the number of bits specified in the count operand. As bits are shifted, zeroes are shifted in on the right.

Format:

- Format: SAL D, Count or SHL D, Count Action:



Destination	Example
register	SHL AL, CL
MM	SHL byte ptr BETA, CL
register	SHL AL, 1
MM	SHL byte ptr BETA, 1

Pointers:

1. The number of bits to be shifted can be specified as a constant of 1 (only a constant value of 1 is allowed) or in register CL. Register CL is used if more than 1 bit is to be shifted.

2. CF always contains the last bit shifted out of the destination operand.

3. OF is undefined if more than 1 bit is shifted. If only 1 bit is shifted, OF is cleared to 0 if the sign bit retains its original value; otherwise, it is set to 1.

Continuation

4. AF is undefined after execution.

5. SAL/SHL can be used to multiply by powers of two (i.e., D * 2CL). For example, if two bits are shifted to the left, the value of the destination operand is multiplied by 4 (22).

6. For an 8-bit operand, when shifting it to the left 8 times or more, the destination operand will result to 0. For a 16-bit operand, when shifting it to the left 16 times or more, the destination operand will result to 0.

7. If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the SAL/SHL instruction to denote the data size of the destination operand.

Example:

MOV AL, 04H

SHL AL, 1

After execution, AL = 08H.

The flags will be: SF = 0, ZF = 0, PF = 0, CF = 0, OF = 0 AF = undefined

0000 0100

0000 1000

Example:

MOV AL, 02H

MOV CL, 03H

SHL AL, CL

After execution, AL = 10H.

The flags will be: SF = 0, ZF = 0, PF = 0, CF = 0, OF = undefined AF = undefined

0000 0010

0000 0100

0000 1000

0001 0000

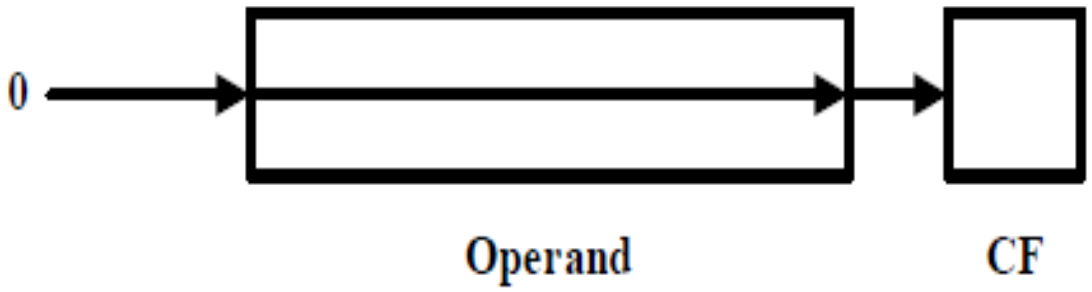
The SHR Instruction

- The SHR (***Shift Logical Right***) instruction shifts to the right the destination operand by the number of bits specified in the count operand. As the bits are being shifted, zeros are shifted in on the left.

Format

Format: SHR D, Count

Action:



Destination	Example
register	SHR AL, CL
MM	SHR byte ptr BETA, CL
register	SHR AL, 1
MM	SHR byte ptr BETA, 1

Pointers:

1. The number of bits to be shifted can be specified as a constant of 1 (only a constant value of 1 is allowed) or in register CL. Register CL is used if more than 1 bit is to be shifted.

2. CF always contains the last bit shifted out of the destination operand.

3. OF is undefined if more than 1 bit is shifted. If only 1 bit is shifted, OF is cleared to 0 if the sign bit retains its original value; otherwise, it is set to 1.

4. AF is undefined after execution.

continuation

5. SHR can be used to divide (unsigned division) by powers of two (i.e. D / 2CL). For example, if two bits are shifted to the right, the value of the destination operand is divided by 4 (22). The result of the division is truncated.

6. For an 8-bit operand, when shifting it to the right 8 times or more, the destination operand will result to 0. For a 16-bit operand, when shifting it to the right 16 times or more, the destination operand will result to 0.

7. If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the SHR instruction to denote the data size of the destination operand.

Example:

MOV AL, 05H

SHR AL, 1

After execution, AL = 02H.

The flags will be: SF = 0, ZF = 0, PF = 0, CF = 1, OF = 0, AF = undefined

0000 0101

0000 0010

Example

MOV AL, 20H

MOV CL, 03H

SHR AL, CL

After execution, AL = 04H.

The flags will be: SF = 0, ZF = 0, PF = 0, CF = 0,
OF = undefined AF = undefined

0010 0000

0001 0000

0000 1000

0000 0100

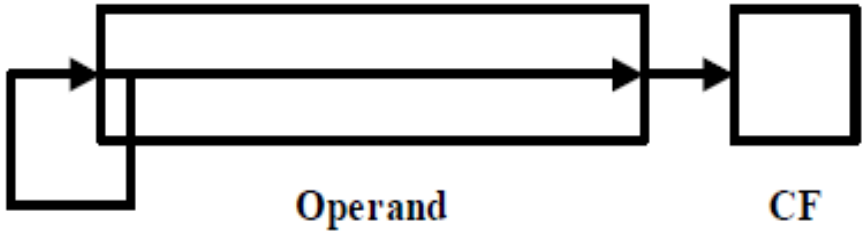
The SAR Instruction

- The SAR (***Shift Arithmetic Right***) instruction shifts to the right the destination operand by the number of bits specified in the count operand. As the bits are being shifted, the most significant bit is shifted in to the left, preserving the sign bit.

Format

Format: SAR D, Count

Action:



Destination	Example
register	SAR AL, CL
MM	SAR byte ptr BETA, CL
register	SAR AL, 1
MM	SAR byte ptr BETA, 1

Pointers:

1. The number of bits to be shifted can be specified as a constant of 1 (only a constant value of 1 is allowed) or in register CL. Register CL is used if more than 1 bit is to be shifted.

2. CF always contains the last bit shifted out of the destination operand.

3. OF is undefined if more than 1 bit is shifted. If only 1 bit is shifted, OF is always cleared to 0. This is because the sign bit is replicated during the shift.

4. AF is undefined after execution.

continuation

5. SAR can be used to divide (signed division) by powers of two (i.e., D / 2CL). For example, if two bits are shifted to the right, the value of the destination operand is divided by 4 (22). The result of the division is rounded down (i.e., 5/2 = 2; -5/2 = -3).

6. For an 8-bit operand, when shifting it to the right 8 times or more, the destination operand will result to the sign extension of the operand. For a 16-bit operand, when shifting it to the right 16 times or more, the destination operand will result to the sign extension of the operand.

7. If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the SAR instruction to denote the data size of the destination operand.

Example

MOV AL, FCH

SAR AL, 1

After execution, AL = FEH.

The flags will be: SF = 1, ZF = 0, PF = 0, CF = 0, OF = 0 AF = undefined

1111 1100

1111 1110 0

Example

MOV AL, 19H

MOV CL, 03H

SAR AL, CL

After execution, AL = 03H.

The flags will be: SF = 0, ZF = 0, PF = 1, CF = 0, OF = undefined; AF = undefined

0001 1001

0000 1100 1

0000 0110 0

0000 0011 0

MODULE 5.2 Part 3

Basic 8086/8088 Instruction Sets: Rotate Instructions

At the end of the lesson, the student should be able to :

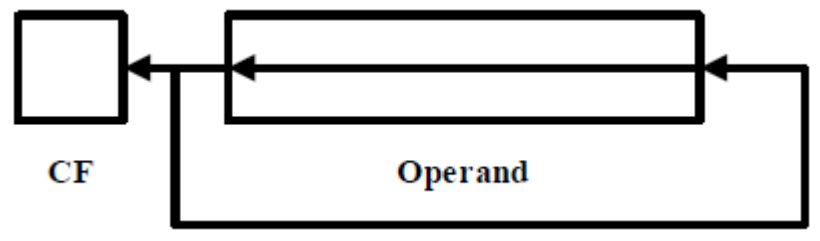
- **Identify the rotate instructions**
- **Identify the values of the conditional flags**

Rotate Instructions

- Rotate instructions rotate bits out into the other end of the operand.
- Rotate instructions can be classified by the direction of the rotation (rotate to the left or rotate to the right). They can also be classified as to whether the carry flag is part of the rotate operation or not.

The ROL Instruction

- The ROL (***Rotate Left***) instruction rotates to the left the destination operand by the number of bits specified in the count operand.
  Format: ROL D, Count
- Action:



Valid Format

Destination	Example
register	ROL AL, CL
MM	ROL byte ptr BETA, CL
register	ROL AL, 1
MM	ROL byte ptr BETA, 1

Pointers:

1. The number of bits to be rotated can be specified as a constant of 1 (only a constant value of 1 is allowed) or in register CL. Register CL is used if more than 1 bit is to be rotated.

2. CF always contains the last bit rotated out of the destination operand.

3. OF is undefined if more than 1 bit is rotated. If only 1 bit is rotated, OF is cleared to 0 if the sign bit retains its original value; otherwise, it is set to 1.

continuation

4. Only CF and OF are affected, the rest are not.

5. For an 8-bit operand, when rotating it to the left 8 times, the value of the destination operand will remain the same. For a 16-bit operand, when rotating it to the left 16 times, the destination operand will remain the same.

6. If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the ROL instruction to denote the data size of the destination operand.

Example

MOV AL, 04H

ROL AL, 1

After execution, AL = 08H.

The flags will be: CF = 0, OF = 0

Example

MOV AL, 55H

MOV CL, 03H

ROL AL, CL

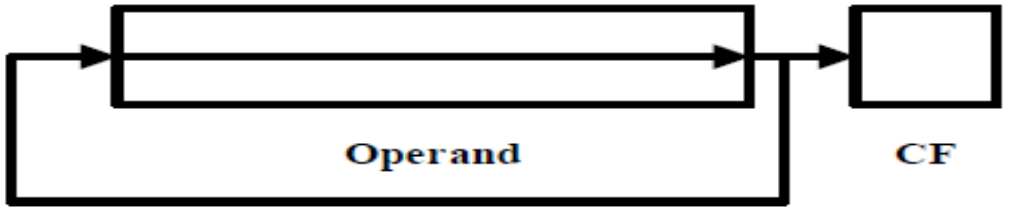
After execution, AL = AAH.

The flags will be: CF = 0; OF = undefined

The ROR Instruction

The ROR (***Rotate Right***) instruction rotates to the right the destination operand by the number of bits specified in the count operand.
Format: ROR D, Count

Action:



Destination	Example
register	ROR AL, CL
MM	ROR byte ptr BETA, CL
register	ROR AL, 1
MM	ROR byte ptr BETA, 1

Pointers:

1. The number of bits to be rotated can be specified as a constant of 1 (only a constant value of 1 is allowed) or in register CL. Register CL is used if more than 1 bit is to be rotated.

2. CF always contains the last bit rotated out of the destination operand.

3. OF is undefined if more than 1 bit is rotated. If only 1 bit is rotated, OF is cleared to 0 if the sign bit retains its original value; otherwise, it is set to 1.

Continuation

4. Only CF and OF are affected, the rest are not.

5. For an 8-bit operand, when rotating it to the right 8 times, the value of the destination operand will remain the same. For a 16-bit operand, when rotating it to the right 16 times, the destination operand will remain the same.

6. If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the ROR instruction to denote the data size of the destination operand.

Example

MOV AL, 04H

ROR AL, 1

After execution, AL = 02H.

The flags will be: CF = 0, OF = 0

Example

MOV AL, 55H

MOV CL, 03H

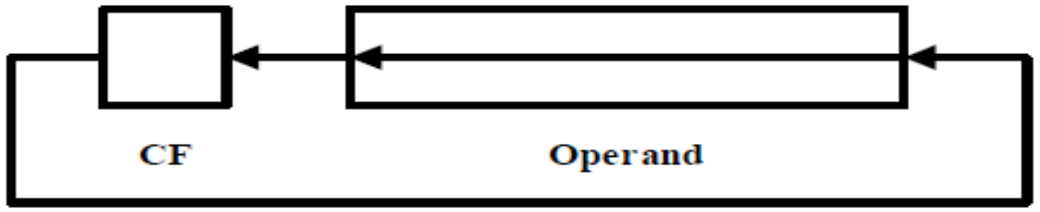
ROR AL, CL

After execution, AL = AAH.

The flags will be: CF = 0; OF = undefined

The RCL Instruction

- The RCL (***Rotate Left through Carry***) instruction rotates to the left the destination operand by the number of bits specified in the count operand. The carry flag is treated as part of the rotate. Format: RCL D, Count
- Action:



Destination	Example
register	RCL AL, CL
MM	RCL byte ptr BETA, CL
register	RCL AL, 1
MM	RCL byte ptr BETA, 1

Pointers:

1. The number of bits to be rotated can be specified as a constant of 1 (only a constant value of 1 is allowed) or in register CL. Register CL is used if more than 1 bit is to be rotated.

2. CF is part of the rotate operation. This means that the CF is rotated into the LSB of the operand and the MSB of the operand goes to CF.

3. OF is undefined if more than 1 bit is rotated. If only 1 bit is rotated, OF is cleared to 0 if the sign bit retains its original value; otherwise, it is set to 1.

continuation

4. Only CF and OF are affected, the rest are not.

5. For an 8-bit operand, when rotating it to the left 9 times, the value of the destination operand will remain the same. For a 16-bit operand, when rotating it to the left 17 times, the destination operand will remain the same.

6. If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the RCL instruction to denote the data size of the destination operand.

Examples:

MOV AL, 04H

RCL AL, 1

Assume that CF = 1.

After execution, AL = 09H.

The flags will be: CF = 0, OF = 0

Example

MOV AL, 55H

MOV CL, 03H

RCL AL, CL

Assume CF = 0. After execution, AL = A9H.

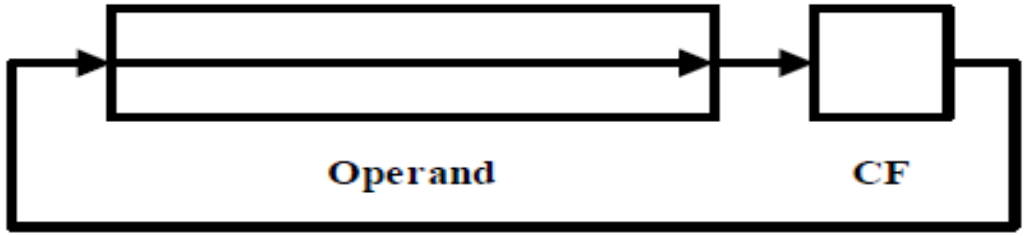
The flags will be:

CF = 0; OF = undefined

The RCR Instruction

The RCR (***Rotate Right through Carry***) instruction rotates to the right the destination operand by the number of bits specified in the count operand. The carry flag is treated as part of the rotate. Format: RCR D, Count

Action:



Destination	Example
register	RCR AL, CL
MM	RCR byte ptr BETA, CL
register	RCR AL, 1
MM	RCR byte ptr BETA, 1

Pointers:

1. The number of bits to be rotated can be specified as a constant of 1 (only a constant value of 1 is allowed) or in register CL. Register CL is used if more than 1 bit is to be rotated.

2. CF is part of the rotate operation. This means that the CF is rotated into the MSB of the operand and the LSB of the operand goes to CF.

3. OF is undefined if more than 1 bit is rotated. If only 1 bit is rotated, OF is cleared to 0 if the sign bit retains its original value; otherwise, it is set to 1.

continuation

4. Only CF and OF are affected, the rest are not.

5. For an 8-bit operand, when rotating it to the right 9 times, the value of the destination operand will remain the same. For a 16-bit operand, when rotating it to the right 17 times, the destination operand will remain the same.

6. If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the RCL instruction to denote the data size of the destination operand.

Example

MOV AL, 04H

RCR AL, 1

Assume that CF = 1.

After execution, AL = 82H.

The flags will be: CF = 0, OF = 1

Example

MOV AL, 55H

MOV CL, 03H

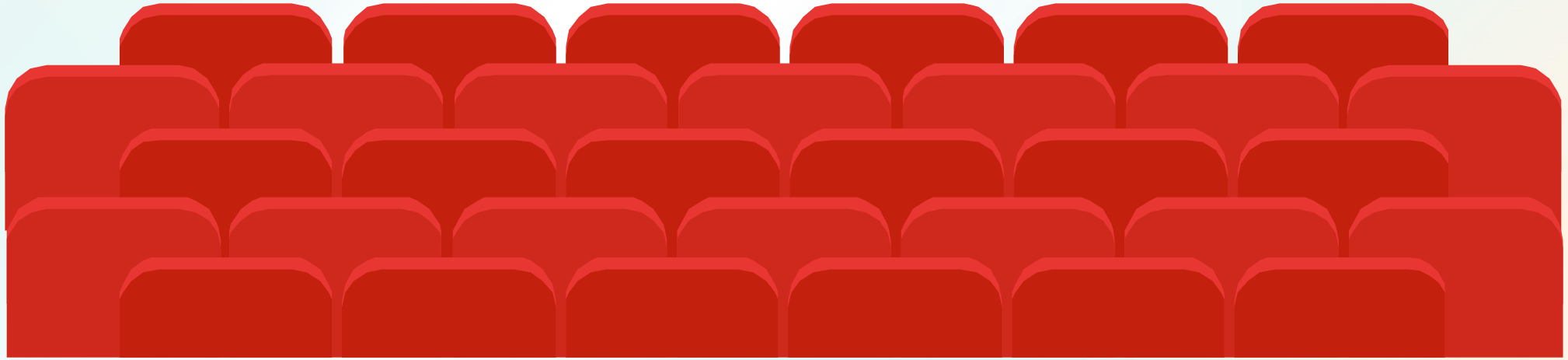
RCR AL, CL

Assume CF = 0.

After execution, AL = 4AH.

The flags will be: CF = 1; OF = undefined

Q&A SESSION

ASK ANY QUESTION RELATED TO OUR TOPIC FOR TODAY.

THANK YOU